CIFAR-10 Image Classification with Convolutional and Residual Networks

1. Introduction

This assignment investigates ten-class image classification on CIFAR-10 [1], whose inputs are 32 x 32 RGB images. The objective is to implement neural networks in PyTorch, compare architectural and training choices, and produce a reproducible model. A fixed split (seed 42) divides the official training set into 45,000 training and 5,000 validation images; the public test set contains 10,000 images. Validation and test images are converted to tensors and normalized with channel-wise mean and standard deviation of 0.5. Checkpoints are selected by validation accuracy before public-test evaluation.

The experiments progress from a prescribed CNN to a batch-normalized CNN and a manually implemented residual network. Batch normalization (BN) [2] and residual learning [3] motivate the architectural changes. All models are trained from scratch using the permitted CIFAR-10 training subset, without pretrained weights or complete architectures imported from model libraries.

2. Methods

Baseline CNN (Exercise 1). Five 5 x 5 convolutional layers use channels 3-8-16-32-64-128 and padding 2. The second and fourth convolutions use stride 2. A 2 x 2 max-pooling layer follows the fifth convolution, yielding 128 x 4 x 4 features. ReLU activations and fully connected layers of sizes 2048-120-84-10 produce unnormalized class scores. The model has 529,742 trainable parameters. Training uses cross-entropy, batch size 64, and SGD with learning rate 0.01 and momentum 0.9 for six epochs.

Convolution shares filters across spatial locations, exploiting local image structure with fewer parameters than dense connections. Successive layers combine local patterns into larger-scale features, while strided convolutions and pooling reduce spatial resolution. ReLU introduces nonlinearity through max(0, x). Cross-entropy penalizes low probability assigned to the correct class; it operates directly on logits using a numerically stable log-softmax internally.

CNN with BN (Exercise 2). The same architecture is extended with BatchNorm2d after each convolution and before activation. The final convolutional sequence is convolution-BN-max pooling-ReLU. Other baseline training settings are retained, with five epochs as required by the exercise. The checkpoint with the highest validation accuracy is restored.

BN standardizes each channel using statistics over the batch and spatial dimensions, then applies a learned scale and shift: y = gamma (x - mean) / sqrt(variance + epsilon) + beta. This controls activation scales and can improve optimization. Training uses batch statistics; evaluation uses accumulated running statistics. Switching between train() and eval() is therefore essential for consistent inference.

Final residual network (Exercise 3). A CIFAR-adapted ResNet-18 is implemented using basic PyTorch layers. A 3 x 3, stride-1 stem produces 64 channels, followed by four stages with channels 64, 128, 256, and 512, each containing two residual blocks. Each block contains two 3 x 3 convolutions with BN; identity shortcuts are replaced by 1 x 1 convolutional projections when dimensions change. The first block of stages 2-4 downsamples with stride 2. Global average pooling and a linear classifier produce ten logits. The model has 11,173,962 trainable parameters and performs no additional input normalization.

A residual block computes ReLU(F(x) + S(x)), where F learns a correction and S is the identity or a projection. The shortcut provides a direct path for information and gradients, making deeper networks easier to optimize. Global average pooling summarizes each channel and reduces the classifier's parameter count relative to flattening the full feature map.

Training-only augmentation combines random cropping with four-pixel padding, horizontal flipping, and Random Erasing (probability 0.25; area fraction 0.02-0.15). The fixed validation split and evaluation preprocessing are unchanged. Training uses batch size 128, SGD with Nesterov momentum 0.9, weight decay 0.0005, and cross-entropy with label smoothing 0.1. A 20-epoch OneCycle schedule warms up over the first 20% of steps, increasing learning rate from 0.01 to 0.1 before annealing to approximately 0.0001. Mixed precision is used on an NVIDIA GeForce RTX 4080 SUPER. Python, NumPy, and PyTorch seeds are set to 42 for this experiment.

Cropping and flipping encourage robustness to position and orientation; erasing discourages dependence on a single visible patch. Weight decay penalizes large weights. Label smoothing assigns target probabilities 0.91 to the correct class and 0.01 to each other class, discouraging overconfident predictions. Learning-rate warm-up moderates initial updates, while later decay enables finer optimization.

3. Experiments and Analysis

3.1 Validation-based comparison

Model	*Epochs*	*Selected epoch*	*Validation accuracy*	*Public-test accuracy*
Baseline CNN	*6*	*6*	*65.84%*	*65.35%*
CNN + BN	*5*	*4*	*73.58%*	*72.85%*
Final residual network	*20*	*20*	***93.62%***	***93.31%***

Adding BN is associated with a 7.74 percentage-point increase in best validation accuracy and a 7.50-point increase in test accuracy. First-epoch training accuracy rises from 14.01% to 44.09%, consistent with faster optimization in these runs. The BN model peaks at epoch 4: validation accuracy falls from 73.58% to 73.10% at epoch 5 while training accuracy increases from 74.69% to 78.61%. Restoring the best validation checkpoint avoids selecting that later epoch.

The final configuration improves best validation accuracy by 20.04 points over CNN + BN, and public-test accuracy by 20.46 points. This comparison supports the effectiveness of the complete configuration. It is not a controlled attribution of individual improvements: training budgets differ, and architecture, augmentation, regularization, and optimization change together. No separate ablations or repeated-seed runs were performed to isolate those contributions or quantify run-to-run variation.

3.2 Training dynamics and class performance

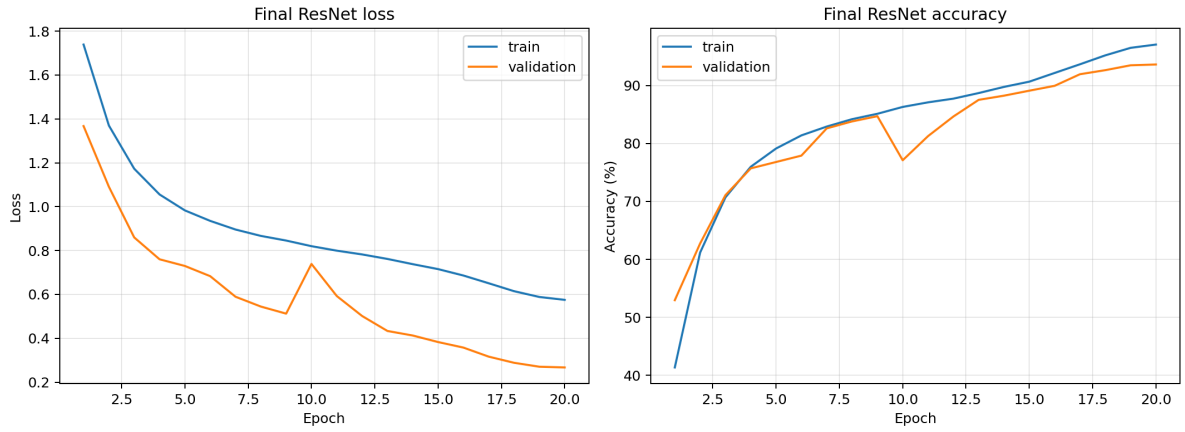


Figure 1. Final-model learning curves over 20 epochs. Training uses augmented inputs and label-smoothed cross-entropy; validation uses unaugmented inputs and ordinary cross-entropy. The loss magnitudes are therefore not directly comparable.

Validation accuracy fluctuates during the higher-learning-rate phase, including a decrease from 84.70% at epoch 9 to 77.10% at epoch 10. It subsequently improves as the learning rate decays, reaching 93.62% at epoch 20. Final training accuracy is 97.06%, versus 93.62% on validation. This 3.44-point difference is descriptive rather than a matched generalization-gap estimate, because training metrics are accumulated during parameter updates on augmented images. The independently loaded submission checkpoint achieves 93.31% on the public test set, close to its validation result.

The final model's per-class public-test accuracies are airplane 94.4%, automobile 96.7%, bird 90.9%, cat 83.8%, deer 95.4%, dog 90.3%, frog 95.7%, horse 94.4%, ship 95.8%, and truck 95.7%. Cat remains the most difficult class, although it improves from 42.5% with CNN + BN to 83.8%. Low-resolution appearance variation and similarity between animal categories are plausible explanations; per-class accuracy alone does not identify which classes are confused. A confusion matrix and inspection of misclassified examples would be needed to verify this hypothesis.

The final checkpoint is approximately 43 MB. Its state dictionary loads strictly into the no-argument build_model() in model.py, and the independent evaluation cell verifies finite outputs of shape [N, 10]. The submitted implementation therefore satisfies the required inference interface while achieving the strongest measured performance of the three configurations.

References

1. A. Krizhevsky. Learning Multiple Layers of Features from Tiny Images. Technical report, 2009.

2. S. Ioffe and C. Szegedy. Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift. ICML, 2015.

3. K. He, X. Zhang, S. Ren, and J. Sun. Deep Residual Learning for Image Recognition. CVPR, 2016.